# Relatório - Analisador Léxico PascaLex

Oct 19, 2026

**Universidade Federal do Espírito Santo (UFES)** Ciência da Computação · Compiladores
Professor: Rodrigo Freitas Silva

**Trabalho 1: Um Analisador Léxico para um dialeto da linguagem Pascal**

Repositório: github.com/leorml/trabalho---lexer IDE online: leorml.github.io/trabalho---lexer

## 1. Sobre o trabalho

A linguagem se chama **PascaLex**, um dialeto de Pascal. O trabalho é um analisador léxico com uma IDE que roda no navegador.

O analisador lê o programa caractere por caractere, ignora espaços e comentários ( `//` , `{ }` e `(* *)` ) e separa o texto em tokens. Ele não diferencia maiúsculas de minúsculas. Identificadores começam com letra e têm até 15 caracteres (letras, números e `_` ).

Quando encontra um erro léxico, a IDE mostra a linha e a coluna e sublinha o trecho de vermelho, inclusive enquanto se digita. Para não parar no primeiro erro, usamos a recuperação em modo pânico: o trecho errado é descartado e a análise continua.

A tabela de símbolos começa com as palavras reservadas, os operadores e a pontuação, e recebe os identificadores do programa. O autômato finito determinístico está no arquivo `automato.png` do repositório e no botão "Ver autômato" da IDE.

## 2. Erros léxicos

| Erro | Exemplo | Mensagem mostrada |
|---|---|---|
| Caractere inválido | `total$ := 1;` | caractere inválido "$" |
| Identificador com mais de 15 caracteres | `nomeDeVariavelMuitoGrande` | identificador "..." passa de 15 caracteres |
| Número mal formado (número grudado em letra) | `12abc` | número mal formado "12abc" |
| Literal sem a aspa de fechamento | `'texto` | literal não foi fechado (falta a aspa) |

| Erro | Exemplo | Mensagem mostrada |
|---|---|---|
| Comentário de chaves sem fechar | `{ ...` até o fim do arquivo | comentário { não foi fechado |
| Comentário de parênteses sem fechar | `(* ...` até o fim do arquivo | comentário (* não foi fechado |

## 3. Tabela de tokens

Na coluna **Valor**, palavras reservadas, operadores e pontuação têm a posição fixa que ocupam na tabela de símbolos. Identificadores recebem a próxima posição livre da tabela (a partir de 46). Números e literais não entram na tabela de símbolos: o valor é o próprio número ou texto.

| Token | Lexema | Expressão regular | Atributo | Valor |
|---|---|---|---|---|
| ID | `soma`, `n1`, `total_2` | `[a-z][a-z0-9_]*` (até 15 caracteres) | entrada na tabela de símbolos | 46 em diante |
| NUM | `10`, `2.5` | `[0-9]+(\.[0-9]+)?` | valor numérico | o número (ex.: 2.5) |
| LITERAL | `'Olá'` | `'[^'\n]*'` | texto | o texto (ex.: 'Olá') |
| PROGRAM | `program` | `program` | nenhum | 1 |
| BEGIN | `begin` | `begin` | nenhum | 2 |
| END | `end` | `end` | nenhum | 3 |
| VAR | `var` | `var` | nenhum | 4 |
| CONST | `const` | `const` | nenhum | 5 |
| PROCEDURE | `procedure` | `procedure` | nenhum | 6 |
| FUNCTION | `function` | `function` | nenhum | 7 |
| INTEGER | `integer` | `integer` | nenhum | 8 |
| REAL | `real` | `real` | nenhum | 9 |
| CHAR | `char` | `char` | nenhum | 10 |
| STRING | `string` | `string` | nenhum | 11 |

| Token | Lexema | Expressão regular | Atributo | Valor |
|---|---|---|---|---|
| IF | `if` | `if` | nenhum | 12 |
| THEN | `then` | `then` | nenhum | 13 |
| ELSE | `else` | `else` | nenhum | 14 |
| WHILE | `while` | `while` | nenhum | 15 |
| DO | `do` | `do` | nenhum | 16 |
| REPEAT | `repeat` | `repeat` | nenhum | 17 |
| UNTIL | `until` | `until` | nenhum | 18 |
| BREAK | `break` | `break` | nenhum | 19 |
| CONTINUE | `continue` | `continue` | nenhum | 20 |
| FOR | `for` | `for` | nenhum | 21 |
| TO | `to` | `to` | nenhum | 22 |
| OU | `or` , `ou` | `or\|ou` | nenhum | 23 e 24 |
| E | `and` , `e` | `and\|e` | nenhum | 25 e 26 |
| ATRIBUICAO | `:=` | `:=` | nenhum | 27 |
| DIFERENTE | `<>` | `<>` | nenhum | 28 |
| MENOR_IGUAL | `<=` | `<=` | nenhum | 29 |
| MAIOR_IGUAL | `>=` | `>=` | nenhum | 30 |
| IGUAL | `=` | `=` | nenhum | 31 |
| MENOR | `<` | `<` | nenhum | 32 |
| MAIOR | `>` | `>` | nenhum | 33 |
| MAIS | `+` | `\+` | nenhum | 34 |
| MENOS | `-` | `-` | nenhum | 35 |
| MULT | `*` | `\*` | nenhum | 36 |
| DIVISAO | `/` | `/` | nenhum | 37 |
| PONTO_VIRGULA | `;` | `;` | nenhum | 38 |
| DOIS_PONTOS | `:` | `:` | nenhum | 39 |
| VIRGULA | `,` | `,` | nenhum | 40 |

| Token | Lexema | Expressão regular | Atributo | Valor |
|---|---|---|---|---|
| PONTO | `.` | `\.` | nenhum | 41 |
| ABRE_PAR | `(` | `\(` | nenhum | 42 |
| FECHA_PAR | `)` | `\)` | nenhum | 43 |
| ABRE_COL | `[` | `\[` | nenhum | 44 |
| FECHA_COL | `]` | `\]` | nenhum | 45 |

Não geram token, e por isso não aparecem na tabela: espaços em branco `[ \t\r\n]+` e comentários `//[^\n]*`, `\{[^}]*\}` e `\(\*` … `\*\)`.